



Balanceadores de carga e ouvintes do ELB

© 2020, Amazon Web Services, Inc. ou uma afilada. Todos os direitos reservados.

Bem-vindo aos Balanceadores de carga e ouvintes do ELB.



O que você aprenderá

Principais pontos da lição

Você aprenderá a:

- Descrever os recursos do Elastic Load Balancing
- Identificar os componentes de uma arquitetura usando um balanceador de carga
- Entender o propósito dos ouvintes e dos grupos de destino
- Analisar os comandos da AWS Command Line Interface (AWS CLI) usados para criar e configurar um balanceador de carga

Principais termos:

- Balanceador de carga
- Listener
- Grupo de destino



Neste módulo, você aprenderá a:

- Descrever os recursos avançados do Elastic Load Balancing
- Identificar os componentes de uma arquitetura que são usados com um balanceador de carga
- Entender o propósito dos ouvintes e dos grupos de destino
- Analisar os comandos da AWS Command Line Interface (AWS CLI) usados para criar e configurar um balanceador de carga

Balanceadores de carga e ouvintes do ELB

Lembre-se das seguintes informações da apresentação do ELB, que resumem os balanceadores de carga:

Pergunte aos alunos: quem se lembra da lição sobre o ELB? Alguém pode explicar o ELB?

Os sites modernos de alto tráfego atendem a centenas de milhares, se não milhões, de solicitações simultâneas de usuários ou clientes. Eles devem retornar o texto, as imagens, o vídeo ou os dados do aplicativo corretos de maneira rápida e confiável. Para escalar de forma econômica para atender a esses altos volumes, as práticas recomendadas de computação moderna geralmente exigem a adição de mais servidores.

Um **balanceador de carga** atua como um *sinizador de tráfego* na frente dos seus servidores, que roteia solicitações de clientes em todos os servidores

Lembre-se das seguintes informações da apresentação do ELB, que resumem os balanceadores de carga:

Pergunte aos alunos: quem se lembra da lição sobre o ELB? Alguém pode explicar o ELB?

Os sites modernos de alto tráfego atendem a centenas de milhares, se não milhões, de solicitações simultâneas de usuários ou clientes. Eles devem retornar o texto, as imagens, o vídeo ou os dados do aplicativo corretos de maneira rápida e confiável. Para escalar de forma econômica para atender a esses altos volumes, as práticas recomendadas de computação moderna geralmente exigem a adição de mais servidores.

Um **balanceador de carga** atua como um *sinizador de tráfego* na frente dos seus servidores, que roteia solicitações de clientes em todos os servidores capazes de atender a essas solicitações. As solicitações são atendidas de forma a maximizar a velocidade e a utilização da capacidade. Ele garante que nenhum servidor esteja sobrecarregado, o que pode prejudicar o desempenho.

Um *balanceador de carga* serve como ponto único de contato para os clientes. O balanceador de carga distribui o tráfego de aplicativos de entrada entre

vários destinos, como instâncias do Amazon Elastic Compute Cloud (Amazon EC2). Esses destinos podem estar em várias zonas de disponibilidade, o que aumenta a disponibilidade do aplicativo.

Você adiciona um ou mais ouvintes ao seu balanceador de carga.

Um *ouvinte* verifica as solicitações de conexão de clientes usando o protocolo e a porta que você configurar. As regras que você define para um ouvinte determinam como o balanceador de carga roteia solicitações para seus destinos registrados. Cada regra consiste em uma prioridade, uma ou mais ações e uma ou mais condições. Quando as condições de uma regra são atendidas, suas ações são executadas. Você deve definir uma regra padrão para cada ouvinte e, opcionalmente, pode definir regras adicionais.

Para obter mais informações, consulte: [O que é um balanceador de carga de aplicativo?](#)

Load balancer: Listeners

- Processo que define a porta e o protocolo nos quais o balanceador de carga escuta.
- Cada balanceador de carga precisa de pelo menos um ouvinte para aceitar o tráfego.
- Até 50 ouvintes podem estar em um balanceador de carga.
- As regras de roteamento são definidas nos ouvintes



4

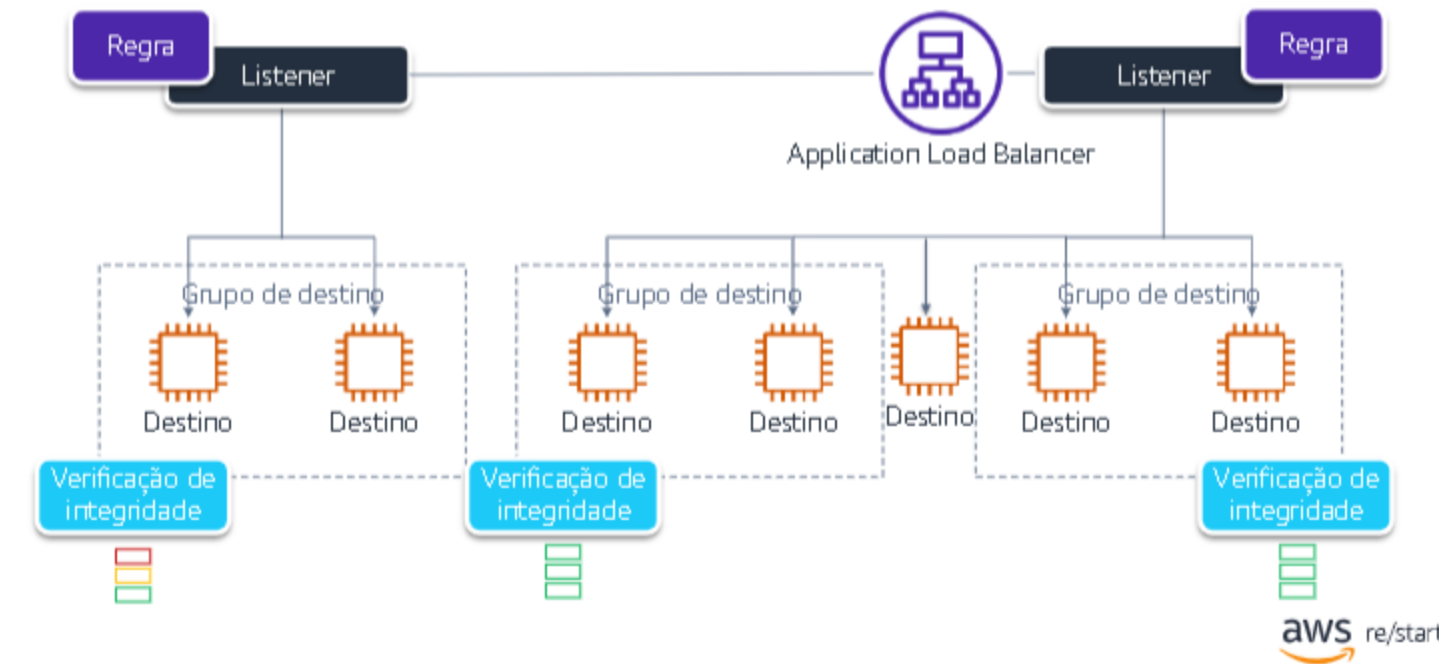
Antes de começar a usar o Application load balancer, você deve adicionar um ou mais listeners.

Um *ouvinte* é um processo que verifica se há uma solicitação de conexão de um cliente para uma instância usando o protocolo e a porta que você especificar.

As *regras* do ouvinte que você definir também determinam como o tráfego de conexão de clientes é roteado pelo balanceador de carga para um ou mais destinos ou grupos de destino.

Balancedor de carga da aplicação

Componentes



Veja um exemplo do balanceador de carga de aplicativo para o site example.com. Um *balanceador de carga* serve como ponto único de contato para os clientes. Ele distribui o tráfego de entrada do aplicativo em vários destinos.

Um Application load balancer pode ter mais de um listener. Este exemplo mostra *dois ouvintes*.

- Cada *ouvinte* verifica as solicitações de conexão de clientes usando o protocolo e a porta que você configurar.
- O listener encaminha solicitações a um ou mais grupos de destino, com base nas regras definidas.

Cada regra especifica um *grupo de destino*, *condição* e *prioridade*.

- Quando a condição é atendida, o tráfego é encaminhado ao grupo de

Veja um exemplo do balanceador de carga de aplicativo para o site `example.com`. Um *balanceador de carga* serve como ponto único de contato para os clientes. Ele distribui o tráfego de entrada do aplicativo em vários destinos.

Um Application load balancer pode ter mais de um listener. Este exemplo mostra *dois ouvintes*.

- Cada *ouvinte* verifica as solicitações de conexão de clientes usando o protocolo e a porta que você configurar.
- O listener encaminha solicitações a um ou mais grupos de destino, com base nas regras definidas.

Cada regra especifica um *grupo de destino*, *condição* e *prioridade*.

- Quando a condição é atendida, o tráfego é encaminhado ao grupo de destino.
- Você deve definir uma regra padrão para cada listener e pode adicionar regras que especificam diferentes grupos de destino com base no conteúdo da solicitação.

- Essa configuração também é conhecida como *roteamento baseado em conteúdo*.

Cada *grupo de destino* roteia solicitações a um ou mais destinos registrados, como instâncias do EC2, usando o protocolo e o número de porta que você especificar.

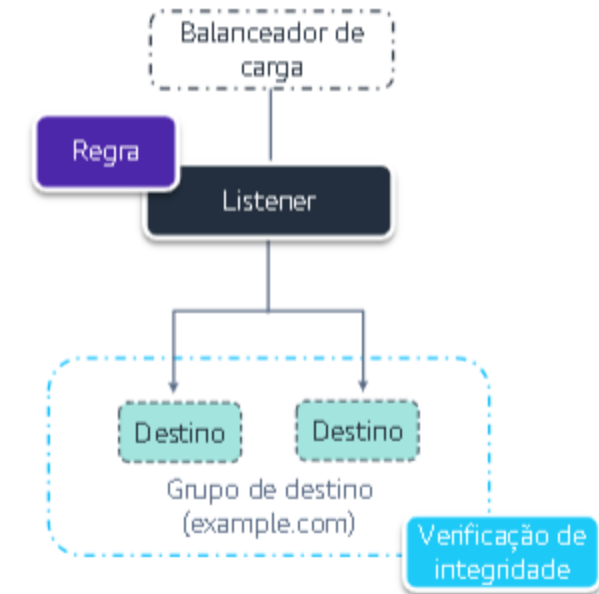
- Você pode registrar um destino com vários grupos de destino.
- Você pode configurar verificações de integridade em cada grupo de destino.

As verificações de integridade são executadas em todos os destinos registrados a um grupo de destino especificado em uma regra de ouvintes para seu balanceador de carga.

Observe que cada listener contém uma regra padrão e um listener contém outra regra que roteia solicitações para um grupo de destino diferente. Como o diagrama indica, você pode registrar um destino com vários grupos de destino.

Load balancer: Grupos de destino

- Grupo de destinos registrados que oferecem suporte a
 - Instâncias do Amazon Elastic Compute Cloud (Amazon EC2)
 - Instâncias de contêiner do Amazon Elastic Container Service (Amazon ECS)
- Um único destino pode ter vários registros de grupo de destino.



aws re/start

Como mencionado anteriormente, um balanceador de carga serve como o único ponto de contato para os clientes. Ao criar cada regra do listener, você especifica um *grupo de destino* e condições. Os destinos são organizados em grupos de destino. Cada grupo de destino é usado para rotear solicitações para um ou mais destinos registrados, como instâncias do EC2 ou instâncias de contêiner do ECS.

O load balancer pode distribuir o tráfego de entrada do aplicativo em vários destinos.

Você também pode configurar verificações de integridde por grupo de destino. Quando o grupo de balanceador de carga especifica um grupo de

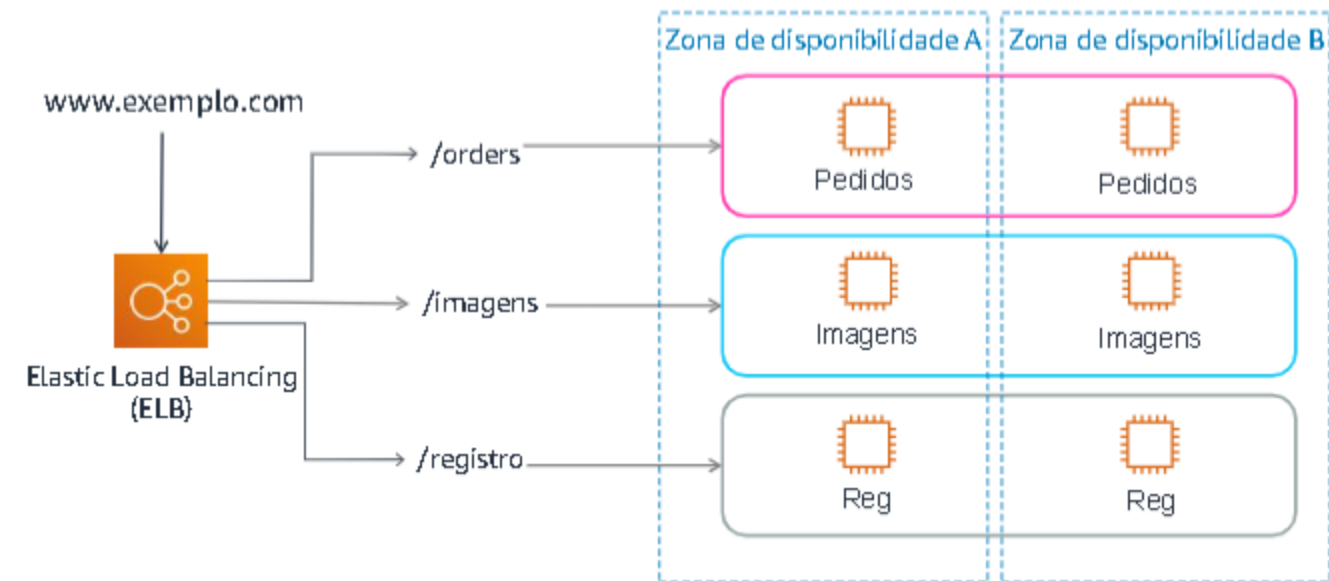
Como mencionado anteriormente, um balanceador de carga serve como o único ponto de contato para os clientes. Ao criar cada regra do listener, você especifica um *grupo de destino* e condições. Os destinos são organizados em grupos de destino. Cada grupo de destino é usado para rotear solicitações para um ou mais destinos registrados, como instâncias do EC2 ou instâncias de contêiner do ECS.

O load balancer pode distribuir o tráfego de entrada do aplicativo em vários destinos.

Você também pode configurar verificações de integridade por grupo de destino. Quando a regra do balanceador de carga especifica um grupo de destino, as verificações de integridade são realizadas em todos os destinos registrados nesse grupo de destino.

Para obter mais informações, consulte [O que é um balanceador de carga de aplicativo?](#)

Application load balancer de exemplo



7

aws re/start

Aqui, você verá um único balanceador de carga de aplicativo configurado com três regras. O design ilustra a funcionalidade de cada segmento, que se baseia no URL solicitado pelo cliente. Os clientes que solicitam a página `www.example.com/orders` são roteados para um conjunto de instâncias do EC2. O cliente que solicita a página `www.example.com/images` é roteado para outras instâncias do EC2. As solicitações para a área de registro são roteadas de forma semelhante para um conjunto diferente de instâncias.

Crie um balanceador de carga de aplicativo: AWS Command Line Interface (AWS CLI)

1. Use o **comando create-load-balancer** para criar um load balancer. Você deve especificar duas sub-redes que não estejam na mesma Zona de disponibilidade.

```
aws elbv2 create-load-balancer \  
--name my-load-balancer \  
--subnets subnet-12345678 subnet-23456789 \  
--security-groups sg-12345678
```



8

Para criar um balanceador de carga de aplicativo usando a AWS Command Line Interface (AWS CLI), conclua as etapas a seguir.

1. Use o comando **create-load-balancer** para criar um balanceador de carga. Você deve especificar duas sub-redes que não estejam na mesma Zona de disponibilidade.

Criar um balanceador de carga de aplicativo: resultado do comando

Comando

```
aws elbv2 create-load-balancer \  
--name my-load-balancer \  
--subnets subnet-12345678 subnet-23456789 \  
--security-groups sg-12345678
```

Resultado

```
{  
  "LoadBalancers": [{  
    "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",  
    "DNSName": "my-load-balancer-1234567890123456.us-east-1.elb.amazonaws.com",  
    "CanonicalHostedZoneId": "Z35SXDOTRQ7X7K",  
    "CreatedTime": "2019-03-28T16:33:59.670Z",  
    "LoadBalancerName": "my-load-balancer", ...  
  ]  
}
```

9



O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{  
  "LoadBalancers": [{  
    "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",  
    "DNSName": "my-load-balancer-1234567890123456.us-east-  
1.elb.amazonaws.com",  
    "CanonicalHostedZoneId": "Z35SXDOTRQ7X7K",  
    "CreatedTime": "2019-03-28T16:33:59.670Z",  
    "LoadBalancerName": "my-load-balancer", ...  
  ]  
}
```



Resultado

```
{
  "LoadBalancers": [{
    "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",
    "DNSName": "my-load-balancer-1234567890123456.us-east-1.elb.amazonaws.com",
    "CanonicalHostedZoneId": "Z35SXDOTRQ7X7K",
    "CreatedTime": "2019-03-28T16:33:59.670Z",
    "LoadBalancerName": "my-load-balancer", ...
```

9



O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{
  "LoadBalancers": [{
    "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",
    "DNSName": "my-load-balancer-1234567890123456.us-east-1.elb.amazonaws.com",
    "CanonicalHostedZoneId": "Z35SXDOTRQ7X7K",
    "CreatedTime": "2019-03-28T16:33:59.670Z",
    "LoadBalancerName": "my-load-balancer",
    "Scheme": "internet-facing",
    "VpcId": "vpc-1234567890123",
```

```
    "State": {
      "Code": "provisioning"
    },
    "Type": "application",
    "AvailabilityZones": [{
      "ZoneName": "us-east-1b",
      "SubnetId": "subnet-12345678"
    },
    {
      "ZoneName": "us-east-1e",
      "SubnetId": "subnet-23456789"
    }
  ],
  "SecurityGroups": [
    "sg-12345678"
  ],
  "IpAddressType": "ipv4"
}]
}
```


Criar um grupo de destino para o load balancer

2. Use o comando **create-target-group** para criar um grupo de destino. Você deve especificar a VPC em que as instâncias do EC2 estão sendo executadas.

```
aws elbv2 create-target-group \  
--name my-targets \  
--protocol HTTP \  
--port 80 \  
--vpc-id vpc-12345678
```



aws re/start

2. Use o comando **create-target-group** para criar um grupo de destino. Você deve especificar a VPC em que as instâncias do EC2 estão sendo executadas.

Criar um grupo de destino para o balanceador de carga: resultado do comando

Comando

```
aws elbv2 create-target-group \  
--name my-targets \  
--protocol HTTP \  
--port 80 \  
--vpc-id vpc-12345678
```

Resultado

```
{  
  "TargetGroups": [{  
    "TargetGroupArn":  
"arn:aws:elasticloadbalancing:us-east-  
1:123456789012:targetgroup/my-targets/9546a5cf349b582c",  
    "TargetGroupName": "my-targets",  
    "Protocol": "HTTP",  
    "Port": 80,  
    "VpcId": "vpc-1234567890123",  
    "HealthCheckProtocol": "HTTP",  
    "HealthCheckPort": "traffic-port",  
    "HealthCheckIntervalSeconds": 30,  
    "HealthCheckTimeoutSeconds": 5,  
    "HealthyThresholdCount": 5,  
    "UnhealthyThresholdCount": 2,  
    "HealthCheckPath": "/",  
    "Matcher": {  
      "HttpCode": "200" ...
```

O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{  
  "TargetGroups": [{  
    "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:targetgroup/my-targets/9546a5cf349b582c",  
    "TargetGroupName": "my-targets",  
    "Protocol": "HTTP",  
    "Port": 80,  
    "VpcId": "vpc-1234567890123",  
    "HealthCheckProtocol": "HTTP",  
    "HealthCheckPort": "traffic-port",  
    "HealthCheckIntervalSeconds": 30,  
    "HealthCheckTimeoutSeconds": 5,  
    "HealthyThresholdCount": 5,  
    "UnhealthyThresholdCount": 2,  
    "HealthCheckPath": "/"
```

O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{  
  "TargetGroups": [{  
    "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:targetgroup/my-targets/9546a5cf349b582c",  
    "TargetGroupName": "my-targets",  
    "Protocol": "HTTP",  
    "Port": 80,  
    "VpcId": "vpc-1234567890123",  
    "HealthCheckProtocol": "HTTP",  
    "HealthCheckPort": "traffic-port",  
    "HealthCheckIntervalSeconds": 30,  
    "HealthCheckTimeoutSeconds": 5,  
    "HealthyThresholdCount": 5,  
    "UnhealthyThresholdCount": 2,  
    "HealthCheckPath": "/",  
    "Matcher": {  
      "HttpCode": "200"  
    },  
    "TargetType": "instance"  
  }]  
}
```

```
healthCheckPort": "traffic-port",  
"HealthCheckIntervalSeconds": 30,  
"HealthCheckTimeoutSeconds": 5,  
"HealthyThresholdCount": 5,  
"UnhealthyThresholdCount": 2,  
"HealthCheckPath": "/",  
"Matcher": {  
  "HttpCode": "200"  
},  
"TargetType": "instance"  
}]
```

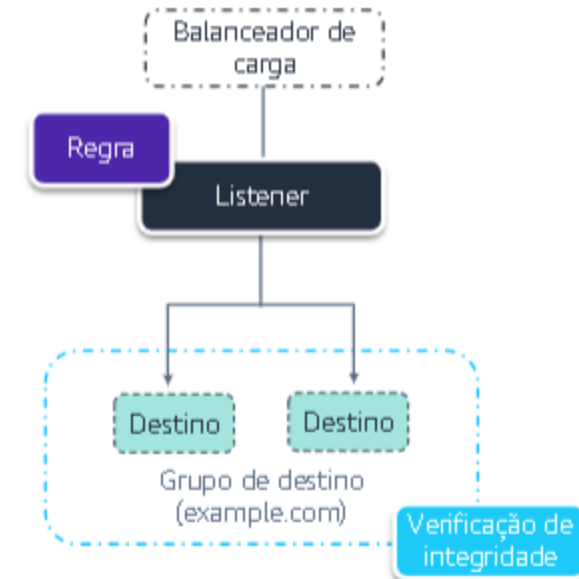
```
}
```

Registrar instâncias do EC2 no grupo de destino

3. Use o comando **register-targets** para registrar suas instâncias com seu grupo de destino.

```
aws elbv2 register-targets \  
--target-group-arn targetgroup-arn \  
--targets Id=i-12345678 Id=i-23456789
```

Esse comando não tem um resultado.



aws re/start

3. Use o comando **register-targets** para registrar suas instâncias com o grupo de destino.

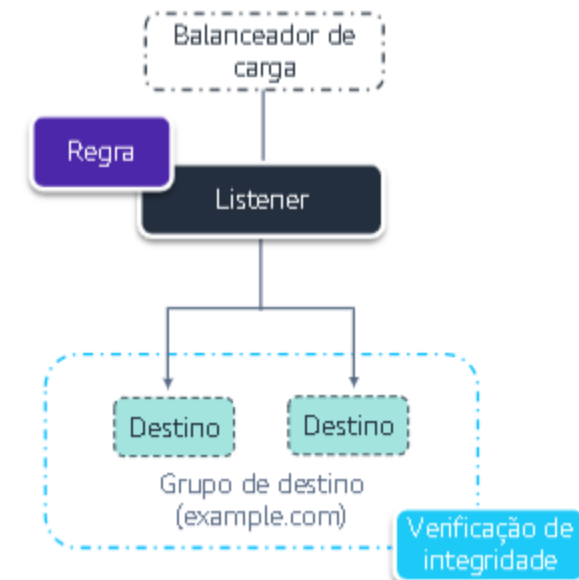
Resultado:

Nenhuma saída de resultado fornecida.

Criar um ouvinte para o balanceador de carga

4. Use o comando **create-listener** para criar um listener para seu load balancer.

```
aws elbv2 create-listener \  
--load-balancer-arn loadbalancer-arn \  
--protocol HTTP \  
--port 80 \  
--default-actions  
Type=forward,TargetGroupArn=targetgroup-arn
```



4. Use o comando **create-listener** para criar um ouvinte para o seu balanceador de carga. O listener tem uma regra padrão que encaminha solicitações para o seu grupo de destino.

Criar um ouvinte para o balanceador de carga: resultado do comando

Comando

```
aws elbv2 create-listener \  
--load-balancer-arn loadbalancer-arn \  
--protocol HTTP \  
--port 80 \  
--default-actions  
Type=forward,TargetGroupArn=targetgroup-arn
```

Resultado

```
{  
  "Listeners": [  
    {  
      "ListenerArn":  
        "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:listener/app/my-load-  
balancer/1234567890123456/14f6f490e5f8bdbf",  
      "LoadBalancerArn":  
        "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:loadbalancer/app/my-load-  
balancer/1234567890123456",  
      "Port": 80,  
      "Protocol": "HTTP",  
      "DefaultActions": [  
        {  
          "Type": "forward",  
          "TargetGroupArn":  
            "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:targetgroup/my-  
targets/9546a5cf349b582c"  
        }  
      ]  
    }  
  ]  
}
```

aws re/start

14

O resultado da emissão do comando deve ser semelhante ao seguinte exemplo:

```
{  
  "Listeners": [  
    {  
      "ListenerArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:listener/app/my-load-  
balancer/1234567890123456/14f6f490e5f8bdbf",  
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",  
      "Port": 80,  
      "Protocol": "HTTP",  
      "DefaultActions": [  
        {  
          "Type": "forward",  
          "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-  
1:123456789012:targetgroup/my-targets/9546a5cf349b582c"  
        }  
      ]  
    }  
  ]  
}
```

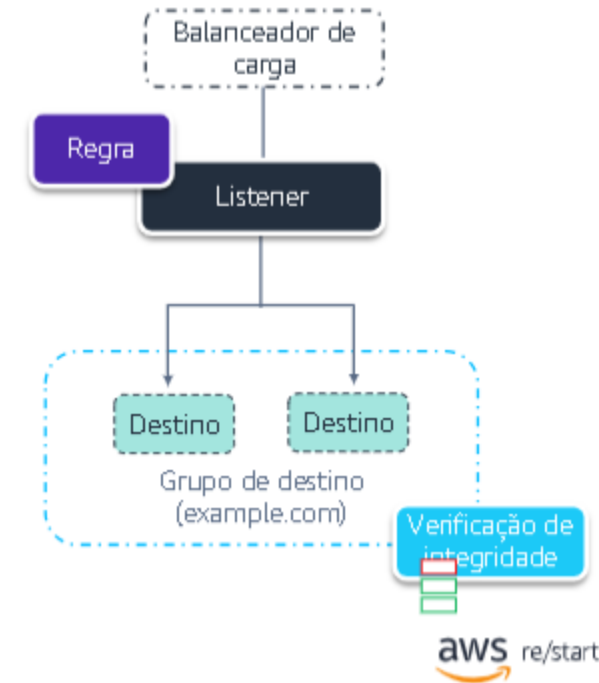
```
{
  "Listeners": [
    {
      "ListenerArn": "arn:aws:elasticloadbalancing:us-east-1:123456789012:listener/app/my-load-balancer/1234567890123456/14f6f490e5f8bdbf",
      "LoadBalancerArn": "arn:aws:elasticloadbalancing:us-east-1:123456789012:loadbalancer/app/my-load-balancer/1234567890123456",
      "Port": 80,
      "Protocol": "HTTP",
      "DefaultActions": [
        {
          "Type": "forward",
          "TargetGroupArn": "arn:aws:elasticloadbalancing:us-east-1:123456789012:targetgroup/my-targets/9546a5cf349b582c"
        }
      ]
    }
  ]
}
```

```
}
]
```


Verificar a integridade dos destinos registrados

5. Opcionalmente, verifique a integridade dos destinos registrados para o seu grupo de destino usando o comando `describe-target-health`.

```
aws elbv2 describe-target-health --target-group-arn targetgroup-arn
```



5. Opcionalmente, verifique a integridade dos destinos registrados para o seu grupo de destino usando o comando **describe-target-health**.

Verifique a integridade dos alvos registrados:
resultado do comando

Comando

```
aws elbv2 describe-target-health --target-group-arn targetgroup-arn
```

Resultado

```
{
  "TargetHealthDescriptions": [
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-12345678",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    },
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-23456789",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    }
  ]
}
```



16

O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{
  "TargetHealthDescriptions": [
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-12345678",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    },
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-23456789",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    }
  ]
}
```

O resultado da emissão do comando deve ser semelhante ao exemplo a seguir:

```
{
  "TargetHealthDescriptions": [
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-12345678",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    },
    {
      "HealthCheckPort": "80",
      "Target": {
        "Id": "i-23456789",
        "Port": 80
      },
      "TargetHealth": {
        "State": "healthy"
      }
    }
  ]
}
```

```
{
  "HealthCheckPort": "80",
  "Target": {
    "Id": "i-12345678",
    "Port": 80
  },
  "TargetHealth": {
    "State": "healthy"
  }
},
{
  "HealthCheckPort": "80",
  "Target": {
    "Id": "i-23456789",
    "Port": 80
  },
  "TargetHealth": {
    "State": "healthy"
  }
}
```

```
}
]
}
}
```

Perguntas sobre o ponto de verificação



Um site está sendo executado muito lentamente. Ele tem muitos subdiretórios: `example.com/login`, `example.com/products`, `example.com/orders` e outros. (Observação: o site está hospedado no momento em uma instância do EC2.)

Como você pode usar um balanceador de carga para melhorar o desempenho do site?



As verificações de integridade se aplicam ao balanceador de carga ou aos servidores de destino?

17



Respostas:

1. Cada um dos subdiretórios pode ser tratado por um conjunto diferente de servidores do EC2. As regras em um balanceador de carga de aplicativo poderiam então ser usadas para rotear o tráfego para os diferentes grupos de servidores do EC2. Além disso, como subdiretórios específicos devem ser dimensionados devido à demanda, somente esses sistemas são dimensionados. Essa restrição pode levar a custos mais baixos, pois somente os servidores necessários serão dimensionados em vez de todo o espaço ocupado.
2. As verificações de integridade são usadas para monitorar servidores de destino onde os balanceadores de carga roteiam o tráfego. Se uma verificação de integridade falhar, o balanceador de carga poderá cancelar o



Respostas:

1. Cada um dos subdiretórios pode ser tratado por um conjunto diferente de servidores do EC2. As regras em um balanceador de carga de aplicativo poderiam então ser usadas para rotear o tráfego para os diferentes grupos de servidores do EC2. Além disso, como subdiretórios específicos devem ser dimensionados devido à demanda, somente esses sistemas são dimensionados. Essa restrição pode levar a custos mais baixos, pois somente os servidores necessários serão dimensionados em vez de todo o espaço ocupado.
2. As verificações de integridade são usadas para monitorar servidores de destino onde os balanceadores de carga roteiam o tráfego. Se uma verificação de integridade falhar, o balanceador de carga poderá cancelar o registro do servidor não íntegro.

Principais conclusões



© 2020, Amazon Web Services, Inc. ou sua afiliada. Todos os direitos reservados.

18

- Cada listener verifica solicitações de conexão de clientes e, em seguida, as encaminha para um ou mais grupos de destino, com base nas regras definidas.
- Cada regra especifica um grupo de destino, uma condição e uma prioridade.

aws re/start

Algumas das principais lições desta seção são:

- Cada ouvinte verifica se há solicitações de conexão de clientes usando o protocolo e a porta configurados. Em seguida, ele encaminha solicitações para um ou mais grupos de destino, com base nas regras definidas.
- Cada regra especifica um grupo de destino, uma condição e uma prioridade.